SIMATS ENGINEERING

ASSESSMENT TOOL 2

PSEUDOCODE WRITING TASK

Course Name	**R Programming**
Course Code	**ITA04**
Course Outcome	**CO1: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. Identify and implement appropriate control structures to solve a programming problem.**
Bloom's Level	**BL3 – BL4**
Assessment Tool Weightage	**20%**
Total Questions / Marks	**10 Questions / 100 Marks (10 marks per question)**
Student Name	**Paleru Dharani Govardhan**
Register Number	**192525280**

Code of Conduct

I, Paleru Dharani Govardhan, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dharani

Instructions

Write structured pseudocode (not R syntax) for each scenario. Each solution identifies inputs, data structures, processing steps, control structures, and outputs. Pseudocode uses language-neutral keywords such as BEGIN, INPUT, IF, FOR, WHILE, OUTPUT, and END.

Q1. Campus Canteen Billing

Bloom's level: BL3 – Vectors | Marks: 10

Problem understanding

- **Inputs:** price[1..n], qty[1..n] for one customer.
- **Data structures:** Numeric vectors price, qty, and itemBill.
- **Processing:** Multiply each item price by its ordered quantity; sum the item bills; apply a 10% discount only when the total exceeds ₹500.
- **Output:** Item-wise bills, subtotal, discount, and final payable amount.

Structured pseudocode

```
BEGIN
  INPUT n
  CREATE vectors price[1..n], qty[1..n], itemBill[1..n]
  FOR i ← 1 TO n
    INPUT price[i], qty[i]
    itemBill[i] ← price[i] × qty[i]
  END FOR
  totalBill ← SUM(itemBill)
  IF totalBill > 500 THEN
    discount ← totalBill × 0.10
  ELSE
    discount ← 0
  END IF
  finalAmount ← totalBill − discount
  OUTPUT itemBill
  OUTPUT "Subtotal: ₹", totalBill
  OUTPUT "Discount: ₹", discount
  OUTPUT "Final payable amount: ₹", finalAmount
END
```

Expected result

The program reports each item's cost, subtotal, applicable discount, and final amount due.

Q2. Student Marks Analyser

Bloom's level: BL3 – Vectors and control structures | Marks: 10

Problem understanding

- **Inputs:** marks for 40 students, each assumed to be in the valid mark range.
- **Data structures:** Numeric vector marks[1..40].
- **Processing:** Compute average, maximum, minimum, fail count, and pass percentage. A mark below 40 is a fail.
- **Output:** Average, highest mark, lowest mark, number of failures, and class performance.

Structured pseudocode

```
BEGIN
  CREATE vector marks[1..40]
  FOR i ← 1 TO 40
    INPUT marks[i]
  END FOR
  averageMark ← SUM(marks) / 40
  highestMark ← MAX(marks)
  lowestMark ← MIN(marks)
  failCount ← 0
  FOR i ← 1 TO 40
    IF marks[i] < 40 THEN
      failCount ← failCount + 1
    END IF
  END FOR
  passCount ← 40 − failCount
  passPercentage ← (passCount / 40) × 100
  IF passPercentage > 80 THEN
    performance ← "Class Performance: Good"
  ELSE
    performance ← "Needs Improvement"
  END IF
  OUTPUT averageMark, highestMark, lowestMark
  OUTPUT "Students failed: ", failCount
  OUTPUT "Pass percentage: ", passPercentage, "%"
  OUTPUT performance
END
```

Expected result

The program reports the class statistics and class-performance message based on the pass percentage.

Q3. Hostel Room Allocation Matrix

Bloom's level: BL3 – Matrices | Marks: 10

Problem understanding

- **Inputs:** Occupancy status for 20 rooms; 1 means occupied and 0 means vacant.
- **Data structures:** A 4 × 5 matrix room[1..4, 1..5], and vector vacantByFloor[1..4].
- **Processing:** Count zeros in each row, sum vacancies, and identify the floor with the largest vacancy count.
- **Output:** Vacancies on every floor, total vacancies, and floor with maximum vacancies.

Structured pseudocode

```
BEGIN
  CREATE matrix room[1..4, 1..5]
  FOR floor ← 1 TO 4
    FOR r ← 1 TO 5
      INPUT room[floor, r]   // 1 = occupied; 0 = vacant
    END FOR
  END FOR
  totalVacant ← 0
  FOR floor ← 1 TO 4
    vacantByFloor[floor] ← 0
    FOR r ← 1 TO 5
      IF room[floor, r] = 0 THEN
        vacantByFloor[floor] ← vacantByFloor[floor] + 1
      END IF
    END FOR
    totalVacant ← totalVacant + vacantByFloor[floor]
    OUTPUT "Floor ", floor, " vacancies: ", vacantByFloor[floor]
  END FOR
  maxVacant ← MAX(vacantByFloor)
  maxFloor ← FIRST INDEX WHERE vacantByFloor = maxVacant
  OUTPUT "Total vacant rooms: ", totalVacant
  OUTPUT "Floor with maximum vacancies: ", maxFloor
END
```

Expected result

The program reports vacancies by floor, total vacant rooms, and the floor with the most vacancies.

Q4. Monthly Electricity Bill Calculator

Bloom's level: BL3 – Decision structures | Marks: 10

Problem understanding

- **Input:** Non-negative units consumed in a month.
- **Data structures:** Numeric variables units, energyCharge, fixedCharge, and totalBill.
- **Processing:** Charge ₹3 per unit for the first 100 units, ₹5 per unit for units 101–200, and ₹8 per unit above 200. Add a fixed charge of ₹50.
- **Output:** Units consumed, energy charge, fixed charge, and total bill.

Structured pseudocode

```
BEGIN
  INPUT units
  IF units < 0 THEN
    OUTPUT "Invalid input: units cannot be negative"
    STOP
  END IF
  IF units <= 100 THEN
    energyCharge ← units × 3
  ELSE IF units <= 200 THEN
    energyCharge ← (100 × 3) + ((units − 100) × 5)
  ELSE
    energyCharge ← (100 × 3) + (100 × 5) + ((units − 200) × 8)
  END IF
  fixedCharge ← 50
  totalBill ← energyCharge + fixedCharge
  OUTPUT "Units consumed: ", units
  OUTPUT "Energy charge: ₹", energyCharge
  OUTPUT "Fixed charge: ₹", fixedCharge
  OUTPUT "Total electricity bill: ₹", totalBill
END
```

Expected result

The program reports the slab-wise energy charge, fixed charge, and total bill.

Q5. Sales Report Using Matrix Operations

Bloom's level: BL4 – Matrices and analysis | Marks: 10

Problem understanding

- **Inputs:** Sales of 3 products across 4 weeks.
- **Data structures:** Matrix sales[1..3, 1..4], vector productTotal[1..3], vector weeklyTotal[1..4], and transposed matrix weekRows[1..4, 1..3].
- **Processing:** Sum each product row and each week column; find maximum totals; transpose the matrix.
- **Output:** Product totals, weekly totals, best-performing product, best week, and transposed matrix.

Structured pseudocode

```
BEGIN
  CREATE matrix sales[1..3, 1..4]
  FOR product ← 1 TO 3
    FOR week ← 1 TO 4
      INPUT sales[product, week]
    END FOR
  END FOR
  FOR product ← 1 TO 3
    productTotal[product] ← SUM(sales[product, 1..4])
  END FOR
  FOR week ← 1 TO 4
    weeklyTotal[week] ← SUM(sales[1..3, week])
  END FOR
  bestProductIndex ← FIRST INDEX OF MAX(productTotal)
  bestWeekIndex ← FIRST INDEX OF MAX(weeklyTotal)
  CREATE matrix weekRows[1..4, 1..3]
  FOR product ← 1 TO 3
    FOR week ← 1 TO 4
      weekRows[week, product] ← sales[product, week]
    END FOR
  END FOR
  OUTPUT "Sales per product: ", productTotal
  OUTPUT "Sales per week: ", weeklyTotal
  OUTPUT "Best-performing product: ", bestProductIndex
  OUTPUT "Best week: ", bestWeekIndex
  OUTPUT "Transposed matrix (weeks as rows): ", weekRows
END
```

Expected result

The program reports product and weekly totals, identifies the best product and week, and displays the transposed matrix.

Q6. Library Book Management Using Lists

Bloom's level: BL3 – Lists and class types | Marks: 10

Problem understanding

- **Inputs:** Details for five books and a title to borrow.
- **Data structures:** A list of five book records; each record contains title, author, year, and availability.
- **Processing:** Display books published after 2015; search for the requested title; if found and available, mark it unavailable. Report if not found or already borrowed.
- **Output:** Recent books and borrowing status.

Structured pseudocode

```
BEGIN
  CREATE list books containing 5 book records
  FOR i ← 1 TO 5
    INPUT books[i].title, books[i].author, books[i].year
    INPUT books[i].availability   // TRUE or FALSE
  END FOR
  OUTPUT "Books published after 2015:"
  FOR i ← 1 TO 5
    IF books[i].year > 2015 THEN
      OUTPUT books[i].title, books[i].author, books[i].year, books[i].availability
    END IF
  END FOR
  INPUT borrowTitle
  found ← FALSE
  FOR i ← 1 TO 5
    IF books[i].title = borrowTitle THEN
      found ← TRUE
      IF books[i].availability = TRUE THEN
        books[i].availability ← FALSE
        OUTPUT "Borrow successful. Book is now unavailable."
      ELSE
        OUTPUT "Book is already borrowed / unavailable."
      END IF
      EXIT LOOP
    END IF
  END FOR
  IF found = FALSE THEN
    OUTPUT "Title not found."
  END IF
END
```

Expected result

The program lists books published after 2015 and updates availability for a successfully borrowed title.

Q7. Temperature Data Cleaning

Bloom's level: BL4 – Vectors, loops, and analysis | Marks: 10

Problem understanding

- **Inputs:** 30 daily temperature readings; some may be missing (NA).
- **Data structures:** Numeric vector temperature[1..30] and character vector category[1..30].
- **Processing:** Count missing values, calculate the mean using only valid readings, replace each NA with that mean, and classify each day.
- **Output:** Missing-value count, cleaned readings, and daily categories.

Structured pseudocode

```
BEGIN
  CREATE vector temperature[1..30]
  FOR day ← 1 TO 30
    INPUT temperature[day]   // value may be NA
  END FOR
  missingCount ← 0
  validSum ← 0
  validCount ← 0
  FOR day ← 1 TO 30
    IF temperature[day] IS NA THEN
      missingCount ← missingCount + 1
    ELSE
      validSum ← validSum + temperature[day]
      validCount ← validCount + 1
    END IF
  END FOR
  IF validCount = 0 THEN
    OUTPUT "Cannot impute: all readings are missing."
    STOP
  END IF
  validMean ← validSum / validCount
  FOR day ← 1 TO 30
    IF temperature[day] IS NA THEN
      temperature[day] ← validMean
    END IF
    IF temperature[day] < 20 THEN
      category[day] ← "Cold"
    ELSE IF temperature[day] <= 30 THEN
      category[day] ← "Moderate"
    ELSE
      category[day] ← "Hot"
    END IF
  END FOR
  OUTPUT "Missing values found: ", missingCount
  OUTPUT "Cleaned temperatures: ", temperature
  OUTPUT "Daily categories: ", category
END
```

Expected result

The program reports missing readings, fills missing values with the valid-data mean, and classifies all days.

Q8. Exam Seating Using 3D Arrays

Bloom's level: BL4 – Three-dimensional arrays | Marks: 10

Problem understanding

- **Inputs:** Register number at each seat; 0 indicates an empty seat.
- **Data structures:** 3D array seat[1..3, 1..4, 1..5] and vector occupiedCount[1..3].
- **Processing:** Traverse each block, row, and column; count non-zero seats; identify the block with the highest count.
- **Output:** Occupied seats per block, block with highest occupancy, and register number at Block 2, Row 3, Column 4.

Structured pseudocode

```
BEGIN
  CREATE 3D array seat[1..3, 1..4, 1..5]
  FOR block ← 1 TO 3
    FOR row ← 1 TO 4
      FOR column ← 1 TO 5
        INPUT seat[block, row, column]
      END FOR
    END FOR
  END FOR
  FOR block ← 1 TO 3
    occupiedCount[block] ← 0
    FOR row ← 1 TO 4
      FOR column ← 1 TO 5
        IF seat[block, row, column] ≠ 0 THEN
          occupiedCount[block] ← occupiedCount[block] + 1
        END IF
      END FOR
    END FOR
    OUTPUT "Block ", block, " occupied seats: ", occupiedCount[block]
  END FOR
  highestCount ← MAX(occupiedCount)
  highestBlock ← FIRST INDEX WHERE occupiedCount = highestCount
  OUTPUT "Block with highest occupancy: ", highestBlock
  OUTPUT "Register number at Block 2, Row 3, Column 4: ", seat[2, 3, 4]
END
```

Expected result

The program reports occupancy by block, the most occupied block, and the requested seat's register number.

Q9. Employee Data Frame Processing

Bloom's level: BL4 – Data frames and class types | Marks: 10

Problem understanding

- **Inputs:** Employee records containing EmpID, Name, Department, and Salary.
- **Data structures:** Data frame employees and numeric column NewSalary.
- **Processing:** Filter IT employees earning above ₹50,000; calculate average salary by department; give a 10% raise only to employees earning below ₹30,000.
- **Output:** Filtered employee records, department-wise average salaries, and updated data frame.

Structured pseudocode

```
BEGIN
  INPUT employee records into data frame employees
  // Columns: EmpID, Name, Department, Salary
  OUTPUT rows WHERE Department = "IT" AND Salary > 50000
  FOR EACH distinct department d in employees.Department
    departmentRows ← rows WHERE Department = d
    averageSalary[d] ← MEAN(departmentRows.Salary)
    OUTPUT d, averageSalary[d]
  END FOR
  CREATE column NewSalary in employees
  FOR i ← 1 TO NUMBER_OF_ROWS(employees)
    IF employees.Salary[i] < 30000 THEN
      employees.NewSalary[i] ← employees.Salary[i] × 1.10
    ELSE
      employees.NewSalary[i] ← employees.Salary[i]
    END IF
  END FOR
  OUTPUT employees including NewSalary
END
```

Expected result

The program displays the requested IT employees, average salary per department, and the updated NewSalary column.

Q10. Prime Number and Fibonacci Series Generator

Bloom's level: BL3–BL4 – Loops and vectors | Marks: 10

Problem understanding

- **Input:** Positive integer N.
- **Data structures:** Vector primes for prime numbers and vector fibonacci for the first N Fibonacci terms.
- **Processing:** Validate N; test each number from 2 through N using a divisor loop and a prime flag; generate Fibonacci terms iteratively.
- **Output:** Prime vector, sum and count of primes, and first N Fibonacci terms.

Structured pseudocode

```
BEGIN
  INPUT N
  WHILE N is not a positive integer
    OUTPUT "Enter a positive integer for N."
    INPUT N
  END WHILE
  CREATE empty vector primes
  FOR number ← 2 TO N
    isPrime ← TRUE
    divisor ← 2
    WHILE divisor × divisor <= number
      IF number MOD divisor = 0 THEN
        isPrime ← FALSE
        EXIT WHILE
      END IF
      divisor ← divisor + 1
    END WHILE
    IF isPrime = TRUE THEN
      APPEND number TO primes
    END IF
  END FOR
  primeCount ← LENGTH(primes)
  IF primeCount > 0 THEN
    primeSum ← SUM(primes)
  ELSE
    primeSum ← 0
  END IF
  CREATE vector fibonacci of length N
  IF N >= 1 THEN fibonacci[1] ← 0 END IF
  IF N >= 2 THEN fibonacci[2] ← 1 END IF
  FOR i ← 3 TO N
    fibonacci[i] ← fibonacci[i − 1] + fibonacci[i − 2]
  END FOR
  OUTPUT "Prime numbers: ", primes
  OUTPUT "Count of primes: ", primeCount
  OUTPUT "Sum of primes: ", primeSum
  OUTPUT "First N Fibonacci terms: ", fibonacci
END
```

Expected result

The program displays all primes up to N, their count and sum, and the first N Fibonacci terms.

Assessment Rubric Reference

The following rubric is included for alignment with the assessment criteria.

Criterion	Weightage	Level 4 – Exemplary
Problem Understanding	20%	Identifies all inputs, outputs, and constraints accurately.
Algorithm Design	30%	Complete, correct, and logically sound solution.
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, and functions/modules.
Pseudocode Structure	10%	Well-structured, clear, consistent format that is easy to follow.
Completeness	20%	Handles all cases with a reasonable and optimized approach.